Neural Networks 201 (2026) 108898

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

PSMEKL: Positional and structural multiple empirical kernel learning for node embedding

Zonghai Zhu [a,*], Xinshuai Wei [a], Huanlai Xing [a], Li Feng [a], De Chen [b], Yuge Xu [c]

[a] School of Computing and Artificial Intelligence, Southwest Jiaotong University, Sichuan Province, Chengdu, 611756, PR China
[b] School of Civil Engineering, Southwest Jiaotong University, Sichuan Province, Chengdu, 611756, PR China
[c] School of Automation Science and Engineering, South China University of Technology, Guangdong Province, 510640, Guangzhou, PR China

ARTICLE INFO

Keywords:
Multi-kernel learning
Graph neural network
Node embedding
Non-attributed graph
Positional and structural information

ABSTRACT

Graph Neural Networks (GNNs) are powerful for processing graph-structured data, where nodes are linked through edges. Generally, incorporating positional and structural information into node embeddings can capture distribution characteristics of nodes. However, traditional GNNs struggle to perfectly capture both types of information simultaneously. Moreover, existing node embedding methods struggle to handle both attributed and non-attributed graphs simultaneously. To this end, we propose Positional and Structural Multiple Empirical Kernel Learning (PSMEKL), a method that enhances node feature representations by incorporating positional and structural information. PSMEKL integrates kernel mapping and community detection to capture both positional relationships and global graph structures, optimizing a dedicated criterion function to generate effective node embeddings for both attributed and non-attributed graphs. This approach enhances the distinction between nodes with different structures while ensuring that nodes with similar connectivity patterns share similar embedding features. Extensive experiments confirm that preprocessing node features with PSMEKL significantly improves GNN performance. The code for PSMEKL is available at https://github.com/ZonghaiZhu/PSMEKL.

1. Introduction

Graph Neural Networks (GNNs) (Wu et al., 2022) are advanced machine learning models designed to process graph-structured data with complex relationships. Unlike traditional neural networks such as Convolutional Neural Networks (CNNs) (Rawat & Wang, 2017), GNNs (Kipf & Welling, 2017) capture both local dependencies and topological structures within graphs, enabling more accurate predictions and comprehensive analysis. As a rapidly evolving technology, GNNs continue to attract significant research interest and find applications across diverse domains.

GNNs excel in fields like medical diagnosis (K. Liang et al., 2023), traffic flow prediction (Ye et al., 2020), stock market analysis (Saha et al., 2022), and recommendation systems (J. Li et al., 2024; J. Zhang et al., 2024), where data can be naturally represented as graphs of nodes and edges. By leveraging graph connectivity, GNNs iteratively propagate and aggregate node features, effectively capturing structural relationships. This enables more expressive node representations and enhances predictive performance.

While GNNs effectively capture local topological structures by aggregating information from connected nodes, they often overlook the feature distribution of nodes before aggregation that is an essential factor in graph learning. Moreover, many real-world graph datasets lack node attributes (non-attributed graphs) (Chen et al., 2022) due to technical constraints or privacy concerns during data collection. This absence of features significantly limits the effectiveness of graph self-supervised learning (SSL) methods (X. Chen et al., 2025), which rely on original node attributes to enhance feature representation during aggregation.

The core design of GNNs is inherently graph-structure-centric, prioritizing topological relationships between nodes over individual node features. As a result, GNNs primarily transmit information through neighboring connections, often neglecting the feature distribution of nodes before aggregation. However, the way node features are allocated plays a crucial role in determining model performance, making their proper representation essential for effective learning.

Fig. 1, visualized with t-SNE (L. J. P. van der Maaten, 2008), demonstrates how node aggregation enhances spatial differentiation by simplifying node distribution. With a fixed adjacency matrix, subfigure (a)

* Corresponding author.
E-mail addresses: zzhu@swjtu.edu.cn (Z. Zhu), wxs@swjtu.edu.cn (X. Wei), hxx@home.swjtu.edu.cn (H. Xing), fengli@swjtu.edu.cn (L. Feng), chende@swjtu.edu.cn (D. Chen), xuyuge@scut.edu.cn (Y. Xu).

https://doi.org/10.1016/j.neunet.2026.108898
Received 15 February 2025; Received in revised form 29 July 2025; Accepted 24 March 2026
Available online 25 March 2026

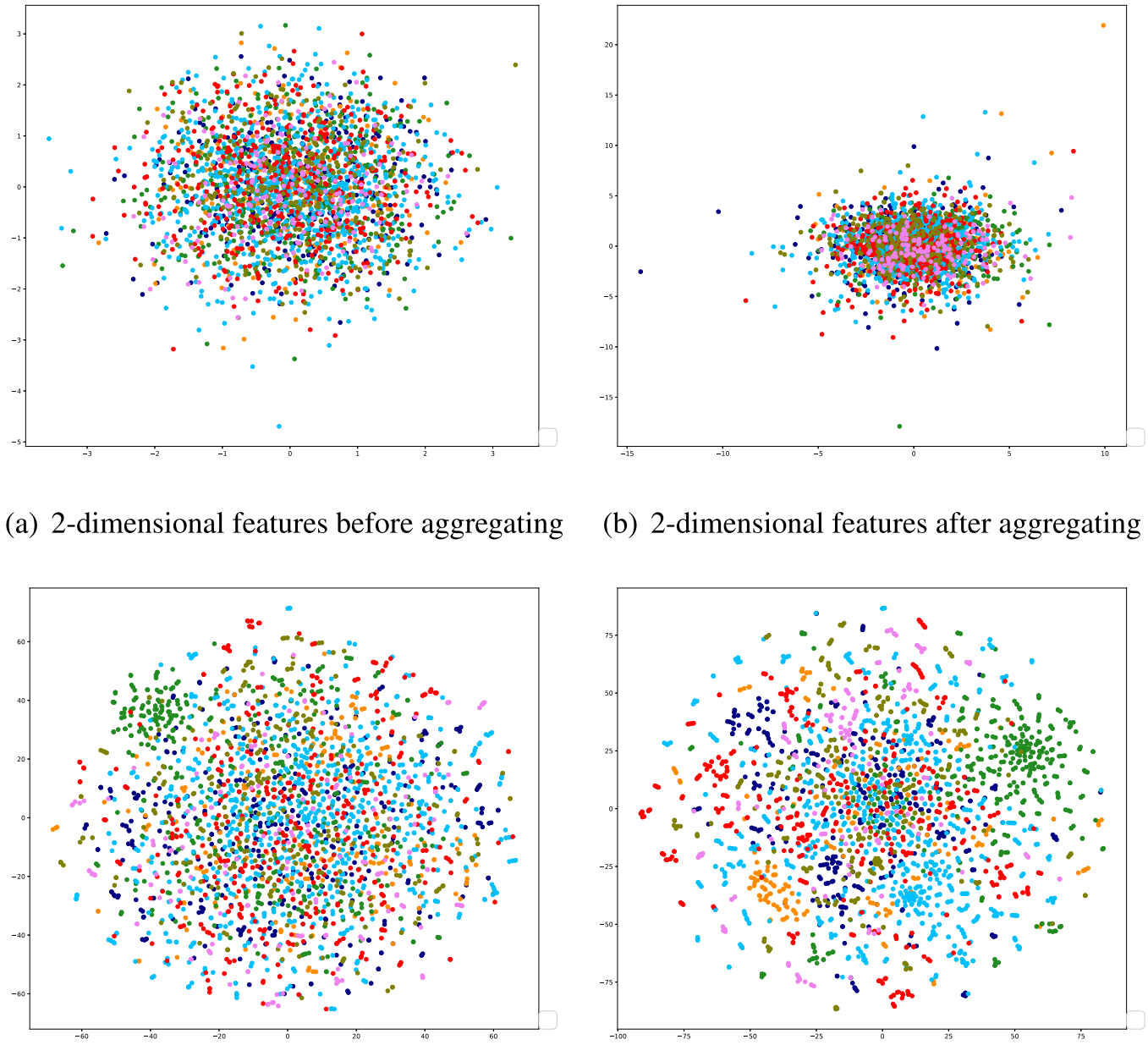


Fig. 1. The adjacency matrix is derived from the CORA dataset. To simplify the complexity of node distribution and make it easier to classify, we increase the feature dimension of nodes whose distribution follows a Gaussian pattern. Since the number of nodes is fixed, the complexity of node distribution decreases as the feature dimensions increase.

presents the original 2D node distribution without aggregation, while subfigures (b) to (d) show the progressive transformation as node features aggregate along link edges. Expanding the feature space increases node separation, making representations more distinguishable. These results emphasize that as node features become more separable, aggregated features also gain clarity, highlighting the crucial role of feature distribution in GNN performance.

Moreover, many real-world graph datasets are non-attributed graphs (X. Liu & Murata, 2021), often due to technical constraints or privacy concerns during data collection. Since self-supervised learning (SSL) methods typically rely on GNN models that depend heavily on node features, non-attributed graphs cannot be directly utilized, limiting SSL effectiveness. While the adjacency matrix can be used as a feature matrix, it results in extremely high-dimensional and sparse representations, posing significant challenges for learning.

As shown in Fig. 1, assigning more reasonable features to the nodes can significantly improve the performance of graph neural networks. Therefore, we aim to explore a method of representing node features before aggregating. This method can enhance the discriminative power of node features and can be applied to both attributed and non-attributed graphs, with the goal of improving the performance of downstream tasks in graph neural networks.

To achieve this, we propose Positional and Structural Multiple Empirical Kernel Learning (PSMEKL), an algorithm based on the principle that nodes with similar positions and structures should share similar features. Using nonlinear kernel mapping, we project the original data into

a high-dimensional kernel space while incorporating graph community structure to assign node features. This ensures that nodes within the same community or with similar connectivity patterns maintain closer feature representations, while enhancing separability for structurally distinct nodes.

For positional information, we employ a kernel function to compute node distances, enabling kernel mapping to separate nonlinear patterns and align connected nodes with closer feature representations. This enhances boundary detection for different node categories in the kernel space. In non-attributed graphs, we use adjacency matrix rows as node features, while in attributed graphs, we directly utilize initial node features. The kernel matrix is then constructed by computing pairwise feature differences. Leveraging Empirical Kernel Mapping (EKM) (Xiong et al., 2005), we project nodes into an explicit kernel space, eliminating the need for high-dimensional sparse adjacency matrices in non-attributed graphs and improving node differentiation.

For community information, the community structure defines global node relationships, grouping closely connected nodes into clusters. Strengthening inter-community distinctions while maintaining intra-community consistency enables GNNs to capture richer structural insights. To achieve this, we employ community detection algorithms (Pizzuti, 2018) to identify graph communities and assign community labels to nodes. By framing a classification task based on these labels, nodes learn structural community information by recognizing whether they belong to the same or different communities. This ensures that nodes within the same community are positioned closer together, while those in different communities remain farther apart.

In the implementation, EKM requires eigen decomposition, and the time complexity increases rapidly as the number of nodes grows. Therefore, we use Multiple Empirical Kernel Mapping (MEKL) (Z. Wang et al., 2019), which partitions the nodes into smaller subsets and performs eigen decomposition within each subset in a more efficient way. Moreover, unlike kernel methods that decompose the kernel matrix and produce high-dimensional node embeddings, PSMEKL can offer node embeddings with a definite feature dimension under an objective function.

The contributions of this study are summarized as follows:

- We propose PSMEKL, an algorithm that effectively provides node embeddings with positional and structural information for both attributed and non-attributed graphs.
- We design a multiple kernel mapping function that utilizes a consistent function to guarantee the agreement of outputs in different kernel spaces and incorporate positional information into node embedding.
- We use community detection algorithms to excavate community structures and integrate structural information into the node embedding process by designing a community classification task.
- We provide the key steps for solving the objective function and show the convergence curve of the loss function. Moreover, experiments validate the proposed PSMEKL in terms of efficiency and effectiveness.

The rest of this paper is organized as follows: Section 2 briefly surveys related work in feature learning for both attributed and non-attributed graphs. Section 3 describes the implementation algorithm named PSMEKL in the proposed learning framework. In Section 4, the experimental results on benchmark graph datasets have shown the feasibility and effectiveness of PSMEKL. Finally, the conclusion is given in Section 5.

2. Related work

In this section, we introduce the methods (C. Wang et al., 2023), including self-supervised learning, node embedding, kernel function, and community detection. Self-supervised learning methods are generally used in attributed graphs, while embedding methods are used in non-attributed graphs. Moreover, the kernel methods and community detection are related to learning positional and structural information.

2.1. Self-supervised learning

Self-supervised learning (SSL) (Liu et al., 2023), which can be generally categorized into generative and contrastive methods, has found widespread adoption in computer vision and natural language processing. In the context of graph learning, SSL has a great achievement on node classification tasks. For example, GCC (Q. Chen et al., 2020) and BGRL (C. Tallec et al., 2022) generally use bi-encoders with momentum updates and exponential average moving to stabilize the training process. Moreover, algorithms such as GCA (Y. Xu et al., 2021) and DGSI (G. Xu et al., 2023) often construct contrastive objectives by using negative sampling. CCA-SSG (Q. Wu et al., 2021) conducted heuristics learning to provide a high-quality graph augmentation.

Except for the contrastive way, self-supervised graph auto encoders (GAEs) directly learn the objective and directly reconstruct the input graph data. Generally, GAEs designed the task related to predicting missing edges or tried to recover vertex features. For example, GraphMAE (X. Liu et al., 2022) reconstructed the node and edge by designing an autoregressive framework. Moreover, algorithm like GATE (H. Davulcu, 2020) mainly focuses on the objectives of link prediction and graph clustering.

2.2. Node embedding algorithms

Node embedding (J. You et al., 2023) primarily serves to endow nodes in graph neural networks with new feature representations. Typically, these graphs contans both attributed and non-attributed graphs, and numerous approaches have sought to enhance node representation capabilities by incorporating positional and structural information (Z. Lu et al., 2022).

The positional node embedding captures node distance information about their relative positions in the graph by mining the connection relationships between nodes. For example, two typical task-independent and unsupervised techniques, including DeepWalk (R. Al-Rfou & Skiena, 2014) and Node2Vec (J. Leskovec, 2016), generate node sequences by performing random walks in the graph to learn the embedding representation of nodes, preserving the local neighbor information of nodes and capturing the semantic associations between similar nodes. β-Random walk (Hirchoua & El Motaki, 2023) improving the random walk for graph embedding by using a node weighting mechanism.

The structural node embedding mainly mines the structural information of nodes in the graph. For example, DDANE (Shen et al., 2023) trains an improved graph-based auto-encoder to provide the embedding with the node attribute and network structure. The degree of nodes reflects the local structural information (Xu et al., 2019), and embedding technology can be used to calculate the value of the degree. The PageRank (Brin & Page, 2012) algorithm can reflect the overall connectivity of the entire graph, and its score can effectively distinguish the importance of different nodes in the graph.

Recently, many methods attempt to incorporate positional or structural information into node embeddings from various perspectives. EigenMLP (Y. Fang et al., 2023) contrasted spatial and spectral views to generate positional and structural features. Node embedding studies (Y. Hu et al., 2024a) (Y. Hu et al., 2024b) (G. Kou et al., 2024) demonstrated measurable performance gains by augmenting random walks with positional and structural information. Moreover, for various graph-based tasks (D. Mandaglio & Tagarelli, 2024) (V. C. Ta, 2024), the introduction of positional and structural node information consistently improves model effectiveness.

2.3. Community detection

Community detection algorithms (Su et al., 2022) can divide these nodes into different communities or groups based on their similarity or connection strength (H. Xing & Xu, 2023). Therefore, community discovery algorithms can effectively find the structural information. For example, the Girvan Newman algorithm (Both et al., 2023) cuts off connections between communities by repeatedly deleting edges in the network and calculates the modularity of each community to evaluate its quality. The Louvain algorithm (Blondel et al., 2008) optimizes community partitioning by maximizing modularity and uses iterative methods based on greedy algorithms for optimization.

The community information can be applied to many fileds. For example, network anomaly detection (Dey et al., 2023) can benefit from the global information of the community. Similarly, by identifying community structure, we can better understand the relationships and interaction patterns in the graph, thereby providing effective support and guidance for node communication, cooperation, and information dissemination within the community.

2.4. Empirical kernel learning

The kernel method (C. You et al., 2022) enriches the expression mode of the sample by mapping the original samples into the kernel space. In the implementation, instead of the general Implicit Kernel Mapping (IKM) (Shawe-Taylor & Cristianini, 2004), we adopt Empirical Kernel Mapping (EKM) that explicitly maps the input data into the kernel space. Given a training set contains N samples and the N samples are defined as $\{(x_i, y_i)\}_{i=1}^{N}$, where y_i is the label of the sample. The symmetrical positive semi-definite kernel matrix is defined as $K = [ker(x_i, x_j)]_{N\times N}$ and

$$ker(x_i, x_j) = exp\left(-\frac{||x_i - x_j||^2}{2\sigma^2}\right) \tag{1}$$

where σ is set to the average value of all l_2 *norm* distances of $||x_i - x_j||_2$

Suppose the rank of K is equal to r, the kernel matrix K is decomposed into:

$$K = \mathrm{Q}_{N\times r}\Lambda_{r\times r}\mathrm{Q}_{N\times r}^{\mathrm{T}} \tag{2}$$

where $\Lambda_{r\times r}$ is a diagonal matrix whose elements are composed of r positive eigenvalues of K, $\mathrm{Q}_{N\times r}$ is the eigenvectors corresponding to r eigenvalues.

To reflect the visualized form in the kernel space, the mapping function is defined as Φ^e ($\Phi^e(\mathcal{I}) \mapsto \mathcal{F}$). For an input sample x in original feature space $\mathcal{I}$, x can be mapped into kernel space $\mathcal{F}$ by Φ^e and $\Phi^e(x)$ is calculated as:

$$\Phi^e(x) = [ker(x, x_1), ker(x, x_2), \ldots, ker(x, x_N)]\mathrm{Q}\Lambda^{-1/2} \tag{3}$$

Kernel mapping uses functions like Gaussian and linear kernels to calculate the similarity or distance between nodes and maps these relationships into a high dimensional kernel space. This process captures the relative positions of nodes, making complex relationships easier to distinguish in the kernel space.

2.5. Relations to our work

Incorporating both positional information and structural information in node embeddings captures distinct critical characteristics of nodes in a network. Positional information focuses on the relative location or distance relationships of nodes within the network, effectively uncovering local relationships between nodes. Structural information, on the other hand, emphasizes the local topological patterns of nodes and the overall topological structure, where nodes with similar structures should also have similar embeddings. Therefore, combining these two aspects enables a more comprehensive representation of network properties.

However, traditional GNNs struggle to perfectly capture both types of information simultaneously, leading to limited performance. Moreover, Existing node embedding methods struggle to handle both attributed and non-attributed graphs simultaneously. Self-supervised approaches for attributed graphs typically rely on node features, making them difficult to apply to non-attributed graphs. Conversely, node embedding methods designed for non-attributed graphs fail to account for scenarios where nodes have attributes.

Unlike existing random walk and self-supervised approaches, PSMEKL learns positional information between nodes from a multi-kernel mapping perspective, while capturing structural information through community structure analysis. Optimized via multi-kernel learning and community classification, PSMEKL jointly learns positional relationships and captures structural information for both attributed and non-attributed graphs.

3. Proposed method

In this section, we introduce the proposed algorithm named PSMEKL in detail. The learning framework of PSMEKL is as follows,

$$\begin{aligned} &L = \alpha L_{posi} + \beta L_{stru}, \\ &s.t.\ \ \alpha + \beta = 1 \end{aligned} \tag{4}$$

L_{posi} is related to positional information of nodes, and it controls how to embed the nodes from multiple kernel spaces into the explicit embedding space. L_{stru} is related to the community classification task, and it controls how to integrate structural information into the node embedding process.

3.1. Multiple kernel learning

The major purpose of multiple kernel learning is to learn the positional relationships between pairs of nodes from kernel space and reduce the computational complexity from $\mathcal{O}(N^3)$ to $\mathcal{O}(N^3/P^2)$, where N and P are the numbers of nodes and subsets, respectively.

By selecting an appropriate kernel function, we implicitly increase the inner product between similar points, reducing distances between connected nodes in the kernel space and strengthening their connectivity. However, decomposing the kernel matrix K has a computational complexity of $\mathcal{O}(N^3)$. To improve efficiency, we partition nodes into P subsets, each containing approximately N/P nodes, allowing us to compute P smaller kernel matrices $\{K_l, l = 1, \ldots, P\}$, significantly reducing computational overhead.

When we obtained P subsets and corresponding P kernel matrices, we can decompose the kernel matrices and adopt EKM to explicitly maps the input data into lth ($l = 1, \ldots, P$) kernel space as follows,

$$\Phi_l^e(x) = [ker_l(x, x_1), \ldots, ker_l(x, x_{N/P})]\mathrm{Q}_l\Lambda_l^{-1/2} \tag{5}$$

where x_i represents the ith row in the adjacency matrix for non-attributed graphs and the ith node feature for attributed graphs.

The engen-decomposition process exhibits an actual time complexity of $O(N^3)$. Therefore, the enormous number of nodes in large-scale graph-structured data results in an extremely large kernel matrix formed by pairwise node comparisons, making eigen-decomposition computationally infeasible. To address this problem, we selectively compute node relations only for nodes with high degrees. Then, these selected nodes are further partitioned into P subsets and each subset contains $N_s \ll N$ nodes. As the number of nodes decreases in each subsets, the computational complexity for eigen-decomposition drops significantly.

3.2. Consistent learning

The major purpose of consistent learning is to establish connections and offer complementary information for kernel spaces and their corresponding kernel mappings, which are built from partitioned node subsets.

We utilize the idea of Canonical Correlation Analysis (CCA) (Li & Shawe-Taylor, 2006) to embed the complementarity provided by the P-view of features into the term L_{posi} to establish relationships between P-views effectively. To make the formula concise, we use matrices X_l to represent the node set $X_l = [\Phi_l^e(x_1)^T, \Phi_l^e(x_2)^T, \dots, \Phi_l^e(x_N)^T]^T$ in the lth kernel space, and the formula of L_{posi} is written as follows,

$$L_{posi} = \frac{1}{N}\sum_{l=1}^{P} tr\left(\left(X_l W_l - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right)\left(X_l W_l - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right)^{\mathrm{T}}\right) \tag{6}$$

The equation shows that node embeddings mapped from different kernel spaces should be in agreement, allowing us to utilize complementary information from each space. Since outputs from different kernel spaces are interrelated, computing the partial derivative of L_{posi} with respect to W_l in the lth kernel space involves W_o $(o \neq l)$ in the other kernel space. The optimization process for W_l in the lth kernel space is as follows,

$$\begin{aligned} dL_{posi} &= \frac{1}{N} tr\left(\left(X_l W_l - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right) d\left(X_l W_l - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right)^{\mathrm{T}}\right) \\ &= \frac{1}{N} tr\left(\left(X_l W_l - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right)\left(dW_l^{\mathrm{T}} X_l^{\mathrm{T}} - \frac{1}{P} dW_l^{\mathrm{T}} X_l^{\mathrm{T}}\right)\right) \\ &= \frac{1}{N} tr\left(\frac{P-1}{P} X_l^{\mathrm{T}} X_l W_l dW_l^{\mathrm{T}} - \frac{P-1}{P^2} X_l^{\mathrm{T}} \sum_{j=1}^{P} X_j W_j dW_l^{\mathrm{T}}\right) \end{aligned} \tag{7}$$

The optimization process for W_l in the other kernel space containing the features matrix X_o mapped by W_o is calculated as follows,

$$\begin{aligned} dL_{posi} &= \frac{1}{N} tr\left(\left(X_o W_o - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right) d\left(X_o W_o - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right)^{\mathrm{T}}\right) \\ &= \frac{1}{N} tr\left(\left(X_o W_o - \frac{1}{P}\sum_{j=1}^{P} X_j W_j\right) d\left(-\frac{1}{P} w_l^{\mathrm{T}} X_l^{\mathrm{T}}\right)\right) \\ &= \frac{1}{N} tr\left(-\frac{1}{P} X_l^{\mathrm{T}} X_o W_o dW_l^{\mathrm{T}} + \frac{1}{P^2} X_l^{\mathrm{T}} \sum_{j=1}^{P} X_j W_j dW_l^{\mathrm{T}}\right) \end{aligned} \tag{8}$$

It is worth to notice that there are $P-1$ other kernel spaces. Therefore, the partial derivative of L_{posi} with respect to W_l can be calculated by accumulating the partial derivatives from all kernel spaces, and the differential form is as follows,

$$\begin{aligned} dL_{posi} &= \frac{1}{N} tr\left(\frac{P-1}{P} X_l^{\mathrm{T}} X_l W_l dW_l^{\mathrm{T}} - \frac{P-1}{P^2} X_l^{\mathrm{T}} \sum_{j=1}^{P} X_j W_j dW_l^{\mathrm{T}}\right) \\ &\quad + \frac{1}{N} tr\left(-\frac{1}{P} X_l^{\mathrm{T}} \sum_{j=1,j\neq l}^{P} X_j W_j dW_l^{\mathrm{T}} + \frac{P-1}{P^2} X_l^{\mathrm{T}} \sum_{j=1}^{P} X_j W_j dW_l^{\mathrm{T}}\right) \\ &= \frac{1}{N} tr\left(\frac{P-1}{P} X_l^{\mathrm{T}} X_l W_l - \frac{1}{P} X_l^{\mathrm{T}} \sum_{j=1,j\neq l}^{P} X_j W_j\right) dW_l^{\mathrm{T}} \end{aligned} \tag{9}$$

Then, we can calculate the trace of the equation and the partial derivative of L_{posi} with respect to W_l as follows,

$$\frac{\partial L_{posi}}{\partial W_l} = \frac{1}{N}\left(\frac{P-1}{P} X_l^{\mathrm{T}} X_l W_l - \frac{1}{P} X_l^{\mathrm{T}} \sum_{j=1,j\neq l}^{P} X_j W_j\right) \tag{10}$$

Finally, the embedding feature x^e for each node x obtained through linear mapping W_l can be written as follows,

$$x^e = \frac{1}{P}\sum_{l=1}^{P} \Phi_l^e(x) W_l \tag{11}$$

Through using kernel function, we obtain the features of nodes mapped by the explicit kernel functions Φ^e, which naturally contain the positions between any pair nodes.

3.3. Community structure learning

The major purpose of community structure learning is to incorporate community structure information into the node embedding process by detecting communities and creating a community classification task that groups nodes into their respective communities. By doing so, groups of nodes in a graph are more densely connected internally than with the rest of the network.

Community discovery algorithms can group nodes into different communities or clusters, representing collections of nodes with similar characteristics, behaviors, or attributes. Clustering nodes helps us better understand their relationships and internal structures. In this work, we use Louvain community detection to identify non-overlapping communities in graph-structured data and assign virtual labels $\mathcal{Y}'$ to nodes based on their communities.

When we obtain the community information, we design a community classification task to predict node associations with community labels in the embedding space, integrating structural information into the node embedding process. The objective function is as follows,

$$L_{stru} = -\sum_{l=1}^{P}\sum_{i=1}^{N} \log\left(\frac{exp(\Phi_l^e(x_i) W_l W_l^g + b^g) y_i'^{\mathrm{T}}}{exp(\Phi_l^e(x_i) W_l W_l^g + b^g)\vec{1}^{\mathrm{T}}}\right) \tag{12}$$

where W_l^g and b^g are the fixed linear mapping matrix and bias, respectively. y_i' is the one-hot form of the community label assigned by the community detection algorithm. $\vec{1}$ represents a row vector whose elements are all equal to 1.

In the calculation process, we can off the function log and exp. Then the objective function L_{stru} can be calculated as follow,

$$L_{stru} = -\sum_{l=1}^{P}\sum_{i=1}^{N} ((\Phi_l^e(x_i) W_l W_l^g + b^g) y_i'^{\mathrm{T}} - \log(exp(\Phi_l^e(x_i) W_l W_l^g + b^g)\vec{1}^{\mathrm{T}})) \tag{13}$$

The optimization process of updating W_l^g and b^g is the same as that in minimizing cross entropy loss. Here, we present how to calculate the partial derivative of L_{stru} with respect to W_l. To update the linear mapping matrix W_l, we differentiate the two sides of the equation and the process is as follows,

$$\begin{aligned} dL_{stru} &= -\sum_{i=1}^{N}\left(d(\Phi_l^e(x_i) W_l W_l^g + b^g) y_i'^{\mathrm{T}} \right. \\ &\quad \left. - \frac{d exp(\Phi_l^e(x_i) W_l W_l^g + b^g)\vec{1}^{\mathrm{T}}}{exp(\Phi_l^e(x_i) W_l W_l^g + b^g)\vec{1}^{\mathrm{T}}}\right) \\ &= -\sum_{i=1}^{N}\left(d(\Phi_l^e(x_i) W_l W_l^g) y_i'^{\mathrm{T}} \right. \\ &\quad \left. - \frac{exp(\Phi_l^e(x_i) W_l W_l^g + b^g) \odot d(\Phi_l^e(x_i) W_l W_l^g)\vec{1}^{\mathrm{T}}}{exp(\Phi_l^e(x_i) W_l W_l^g + b^g)\vec{1}^{\mathrm{T}}}\right) \end{aligned} \tag{14}$$

Then, we calculate the trace of matrices on both sides of the equation,

$$\begin{aligned} dL_{stru} &= tr\left(-\sum_{i=1}^{N}\left(d(\Phi_l^e(x_i) W_l W_l^g) y_i'^{\mathrm{T}} \right.\right. \\ &\quad \left.\left. - \frac{\vec{1}^{\mathrm{T}}(exp(\Phi_l^e(x_i) W_l W_l^g + b^g) \odot d(\Phi_l^e(x_i) W_l W_l^g))}{exp(\Phi_l^e(x_i) W_l W_l^g + b^g)\vec{1}^{\mathrm{T}}}\right)\right) \\ &= tr\left(-\sum_{i=1}^{N} W_l^g y_i'^{\mathrm{T}} \Phi_l^e(x_i) dW_l \right. \\ &\quad \left. + W_l^g \frac{exp(\Phi_l^e(x_i) W_l W_l^g + b^g)^T}{exp(\Phi_l^e(x_i) W_l W_l^g + b^g)\vec{1}^{\mathrm{T}}} \Phi_l^e(x_i) dW_l\right) \end{aligned} \tag{15}$$

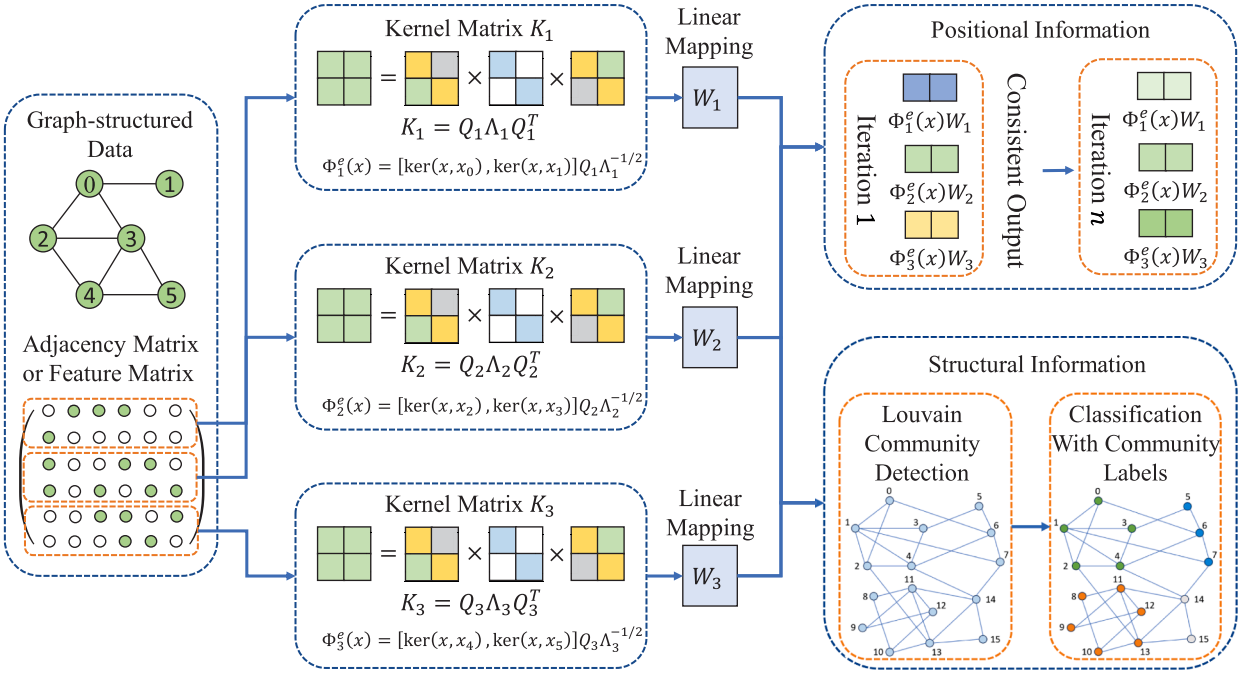


Fig. 2. The framework of PSMEKL works as follows: First, randomly divide the nodes into multiple subsets and calculate the kernel function. Then, calculate the features in each kernel space and map these features to the embedding space, where a consistency term ensures that node embeddings from different kernel spaces align. Additionally, the structural information module extracts and incorporates community structure information into the node embedding by creating a community classification task. (The adjacency matrix and feature matrix are prepared for non-attributed and attributed graphs, respectively.)

The partial derivative of L_{stru} with respect to W_l can be calculated and the specific form of partial derivatives is as follows,

$$\frac{\partial L_{stru}}{\partial W_l} = -\sum_{i=1}^{N} \Phi_l^e(x_i)^T \left(\frac{exp(\Phi_l^e(x_i)W_l W_l^g + b^g)}{exp(\Phi_l^e(x_i)W_l W_l^g + b^g)\vec{1}^{\mathrm{T}}} - y_i \right) W_l^{gT} \tag{16}$$

Finally, we can calculate the partial derivative of L with respect to each $W_l, l = 1, \ldots, P$ as follows,

$$\frac{\partial L}{\partial W_l} = \alpha \frac{\partial L_{posi}}{\partial W_l} + \beta \frac{\partial L_{stru}}{\partial W_l} \tag{17}$$

When we obtain the partial derivative, we can update W_l that connects the lth kernel space. The remaining parameters W_g^l and b_g^l are consistent with the derivative process in the standard Softmax function. The pseudo code of PSMEKL is listed in **Algorithm I**, and the framework of PSMEKL is shown in Fig. 2.

3.4. Analysis of proposed PSMEKL

In graph data, node relationships can be represented using adjacency matrices, Laplacian matrices, or specific kernel functions. Kernel methods transform data into a high-dimensional feature space, making originally similar points even closer together. This property enhances the connectivity of related nodes in graphs.

For given input data points x_i and x_j, kernel methods use a nonlinear mapping $\phi : \mathbb{R}^d \to \mathcal{H}$ to project them into a high-dimensional feature space $\mathcal{H}$, where similarity is computed using the kernel function as follows,

$$k(x_i, x_j) = < \phi(x_i), \phi(x_j) > \tag{18}$$

For points that are close in the original space or connected in the graph, kernel methods ensure that their inner product remains large:

$$k(x_i, x_j) \geq k(x_i, x_k), \quad if\ x_i\ and x_j\ are\ closely\ related. \tag{19}$$

This means that in the high-dimensional space, originally similar points become even closer. Given an graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ with adjacency matrix $\mathcal{A}$ converted from $\mathcal{E}$,where $\mathcal{V}$ is the set of nodes and $\mathcal{E}$ is the set of edges. The general graph Laplacian matrix is defined as follows,

$$L = D - A \tag{20}$$

where D is the degree matrix. If we use w_{ij} represents the ith row and jth column value in L, according to spectral graph theory, minimizing the following embedding objective as follows,

$$min \sum_{(i,j)\ inE} w_{ij} ||\phi(x_i) - \phi(x_j)|| \tag{21}$$

The above equation ensures that the high-dimensional node representations preserve the original graph structure. Then, expanding the

Algorithm 1: Learning algorithm of PSMEKL.

Input: Graph-structured data $\mathcal{G}(\mathcal{A}, \mathcal{Y}, \mathcal{X}), \mathcal{A} \in R^{N\times N}$
Parameter: Embedding dim d, embedding layer t, number of subsets P, hyper-parameter α and β, and embedding learning rate η.
Output: Embedding matrix X^e
Calculate $\mathcal{X} = \mathcal{A}^t$ and $\mathcal{X} = \mathcal{A}^t\mathcal{X}$ for non-attributed and attributed graphs.
Calculate partial kernel matrices $K_l, l = 1, ., P$ on P subsets.
Calculate $\Phi_l^e, l = 1, \ldots, P$ and map corresponding nodes.
Obtain $|\mathcal{Y}'|$ communities with labels $\mathcal{Y}'$ by Louvain method.
Initialize linear mapping $W_l \in R^{d_l\times d}, l = 1, ., P$ and each element of $W_l \sim U(0.5, 1.5)$.
Random Initialize $W_l^g \in R^{d\times|\mathcal{Y}'|}$, $b^g \in R^{|\mathcal{Y}'|}$
while iterations ≤ 100 **do**
  Calculate loss $L = \alpha L_{posi} + \beta L_{stru}$.
  Update $W_l, W_g^l, b_g^l, l = 1, ., P$ by minimizing loss with learning rate η.
end while
Calculate $x_i^e = \frac{1}{P}\sum_{l=1}^{P} \Phi_l^e(x_i)W_l, i = 1, \ldots, N$.
return $X^e = [x_1^{eT}, \ldots, x_N^{e\ T}]^T \in R^{N\times d}$.

equation as follows,

$$\sum_{(i,j)\in E} w_{ij}||\phi(x_i)-\phi(x_j)|| = \sum_i \phi(x_i)^T D\phi(x_i) - \sum_{(i,j)\in E} w_{ij}\phi(x_i)^T\phi(x_j)$$
$$= Tr(\Phi^T L\Phi) \tag{22}$$

where Φ is the feature matrix after mapping. Therefore, the kernel mapping implicitly increases the inner product between similar points, making their distances smaller in the high-dimensional space, thereby enhancing their connectivity.

We select the leading eigenvalues of the kernel matrix, as spectral clustering theory suggests that the first few eigenvectors form more compact clusters. By applying empirical kernel mapping, the high-dimensional feature space is projected into a lower-dimensional representation, ensuring that similar nodes remain closer together.

Community detection algorithms identify groups of nodes in a graph that are more densely connected internally than with the rest of the network. By clustering strongly connected nodes together, these algorithms enhance the structural compactness of related nodes in several ways. Mathematically, community detection seeks to minimize the objective as follows,

$$min \sum_{(i,j)\ inE_{inter}} w_{ij} - \sum_{(i,j)\ inE_{intra}} w_{ij} \tag{23}$$

Therefore, the compactness of each detected community is increased by optimizing intra-cluster edge density. After node embedding, community-related nodes are more compact in the feature space, reinforcing their local structure.

4. Experiment

In this section, we validate the effectiveness and efficiency of the proposed PSMEKL through extensive experiments on both non-attributed and attributed graphs. The detailed description of datasets can be seen in Table 1, and the detailed hyper-parameters settings and description can be seen in Table 3. Moreover, we validate PSMEKL on large-scale graph datasets Table 2, and the detailed hyper-parameters settings shown in Table 4. Parameters not listed in Table 4 follow the same configuration as specified in Table 3. Computations for the experiment are performed on an Intel i5 6600K, Nvidia Titan RTX, and 32GB memory. In the experiment, we will answer the following questions:

- **Classification Performance on non-attributed graphs**: Could PSMEKL outperform existing node embedding algorithm on non-attributed graphs?
- **Ablation study by analyzing hyper-parameters α and β**: What is the impact of positional and structural information of PSMEKL?
- **Training Efficiency of node embedding on non-attributed graphs**: Could PSMEKL have advantages in pre-processing speed in calculating node embedding on non-attributed graphs?
- **Performance on attributed graphs**: Could PSMEKL outperform classical self-supervised or attribute-masked algorithms on attributed graphs?
- **Visualized Results of PSMEKL**: What are the detailed explanations and visualizations of the kernel mapping and community detection processes?

Used Datasets: We compare the classification results of all algorithms on 8 widely used datasets. In these datasets, Plantoid is paper citation graphs, including CORA, CiteSeer, and PubMed. The American air-traffic network, the Brazilian air-traffic network, and the European air traffic network predict the flow level of that node solely based on the structure of the air-traffic network. Moreover, Wikipedia is a cooccurrence network in the first million bytes of the Wikipedia dump. BlogCatalog is a network of social relationships listed on the BlogCatalog website.

Table 1
Summary statistics of non-attributed and attributed graph datasets. (s) stand for single label.

Name	Nodes	Edges	Dimensions	Classes	Attributed
CORA	2708	5429	1433	7 (s)	True
CiteSeer	3327	4732	3703	6 (s)	True
PubMed	19,717	44,338	500	3 (s)	True
Brazil-air	131	1074	NA	4 (s)	False
Europe-air	399	5995	NA	4 (s)	False
USA-air	1190	13,599	NA	4 (s)	False
Wikipedia	2405	17,981	NA	17 (s)	False
BlogCatalog	10,312	667,966	NA	38 (s)	False

Table 2
Summary statistics of large-scale graph datasets. The 'm' and 's' stand for multi-label and single label, respectively.

Name	Nodes	Edges	Dimensions	Classes	Train/Val/Test
PPI	14,755	225,270	50	121 (m)	0.66 / 0.12 / 0.22
Flickr	89,250	899,756	500	7 (s)	0.50 / 0.25 / 0.25
Reddit	232,965	11,606,919	602	41 (s)	0.66 / 0.10 / 0.24
Yelp	716,847	6,977,410	300	100 (m)	0.75 / 0.10 / 0.15
Amazon	1,598,960	132,169,734	200	107 (m)	0.85 / 0.05 / 0.10

Table 3
Hyper-parameter setting of PSMEKL on the experiments. α and β, subject to $\alpha+\beta=1$, stand for the hyper-parameters in Eq. (17), respectively.

Hyper-parameter	values	Hyper-parameter	values
number of subsets	{3, 4, 5}	embedding learning rate	{0.1, 1}
embedding dim	{128, 256, 512}	GNN learning rate	$\{10^{-3}, 10^{-4}\}$
embedding layers	{1, 2}	GNN weight decay	10^{-4}
α	{0, 0.5, 1}	GNN dropout ratio	0
β	{0, 0.5, 1}	GNN layers	{2}, {10, 15}

Table 4
Hyper-parameter setting of PSMEKL on large-scale graph datasets.

Hyper-parameter	PPI	Flickr	Reddit	Yelp	Amazon
number of subsets	4	5	5	3	2
number of nodes per subset	4096	4096	4096	2048	2048
embedding dim	128	128	128	128	128
learning rate	0.001	0.01	0.001	0.001	0.001
weight decay	10^{-5}	10^{-2}	10^{-5}	10^{-5}	10^{-5}

Among these datasets, Plantoid-related datasets are already divided into training set, validation set, and testing set. We follow the public data splits of Cora, Citeseer, and PubMed. For the rest datasets, we select 20%, 40%, and 40% nodes in each class as training, validation, and testing nodes. Classification accuracy $Acc\%$ is adopted here for evaluation.

Comparison Methods on Non-attributed Graphs: We have selected 8 node embedding methods as comparison algorithms. Kernel-based algorithms include EKM and MEKL. EKM reflects the connection differences between nodes and is the foundation of PSMEKL. Therefore, EKM serves as the baseline. MEKL accelerates the mapping process of EKM by partitioning subsets. Random walk related algorithms include Deepwalk (R. Al-Rfou & Skiena, 2014) and Node2vec (J. Leskovec, 2016). Deepwalk learns node feature representations by simulating uniform random walks, and Node2vec is the modification of Deepwalk. TAWE (Y. Hu et al., 2024b) and RARW (G. Kou et al., 2024) introduced positional and structural information based on DeepWalk and Node2Vec. Degree, as a structure-based algorithm, embeds the value of degree into feature space. Moreover, eigen decomposes the adjacency matrix and uses the eigenvectors corresponding to the large eigenvalues as node features.

Comparison Methods on Attributed Graphs: We selected four self-supervised learning algorithms including BGRL, CCA-SSG, and GraphMAE with the GCN. The settings of these algorithms follow the settings

Table 5
Acc (%) results for algorithms using GCN are reported here (The best result on each data set is written in bold). Among the comparison, EKM is served as the baseline.

Dataset	PSMEKL	EKM	MEKL	Deepwalk	Node2vec	TAWE	Degree	RARW	Eigen
CORA	**71.9**$_{\pm 0.9}$	69.5$_{\pm 1.5}$	69.2$_{\pm 1.9}$	69.0$_{\pm 1.5}$	68.3$_{\pm 1.7}$	69.3$_{\pm 1.3}$	65.5$_{\pm 2.1}$	68.2$_{\pm 1.7}$	69.3$_{\pm 0.9}$
CiteSeer	**50.7**$_{\pm 1.7}$	47.5$_{\pm 2.0}$	47.9$_{\pm 0.9}$	47.5$_{\pm 1.6}$	48.0$_{\pm 1.3}$	47.2$_{\pm 1.8}$	44.9$_{\pm 1.6}$	47.4$_{\pm 2.4}$	48.7$_{\pm 1.1}$
PubMed	**69.6**$_{\pm 1.7}$	68.6$_{\pm 1.0}$	67.2$_{\pm 1.2}$	62.8$_{\pm 1.2}$	62.2$_{\pm 1.6}$	62.5$_{\pm 2.0}$	63.0$_{\pm 1.3}$	61.5$_{\pm 1.4}$	68.0$_{\pm 1.5}$
Brazil-air	39.1$_{\pm 4.6}$	40.4$_{\pm 3.4}$	40.0$_{\pm 4.4}$	35.6$_{\pm 6.0}$	35.1$_{\pm 5.9}$	40.8$_{\pm 3.2}$	40.0$_{\pm 5.0}$	**41.7**$_{\pm 4.7}$	37.9$_{\pm 3.1}$
Europe-air	29.6$_{\pm 6.0}$	25.9$_{\pm 9.3}$	22.8$_{\pm 10.3}$	25.4$_{\pm 5.0}$	23.6$_{\pm 6.5}$	**30.0**$_{\pm 4.0}$	21.2$_{\pm 7.8}$	29.5$_{\pm 7.0}$	27.2$_{\pm 8.6}$
USA-air	54.0$_{\pm 1.3}$	54.1$_{\pm 1.0}$	54.9$_{\pm 0.9}$	50.6$_{\pm 1.4}$	50.7$_{\pm 1.3}$	**55.1**$_{\pm 1.1}$	53.3$_{\pm 0.9}$	54.7$_{\pm 1.6}$	49.3$_{\pm 1.4}$
Wikipedia	44.5$_{\pm 0.8}$	44.3$_{\pm 0.9}$	**45.0**$_{\pm 1.0}$	44.5$_{\pm 0.7}$	43.6$_{\pm 0.9}$	42.8$_{\pm 1.0}$	43.0$_{\pm 0.9}$	42.9$_{\pm 0.8}$	43.7$_{\pm 0.8}$
BlogCatalog	23.3$_{\pm 0.6}$	25.8$_{\pm 0.7}$	**26.4**$_{\pm 0.8}$	22.3$_{\pm 0.9}$	22.6$_{\pm 0.6}$	21.0$_{\pm 0.6}$	21.4$_{\pm 1.6}$	21.6$_{\pm 0.6}$	21.5$_{\pm 0.6}$
Average	**47.8**	47.0	46.7	44.7	44.3	46.1	44.0	45.9	45.7

Table 6
Acc (%) results for algorithms using GraphSAGE are reported here (The best result on each data set is written in bold). Among the comparison, EKM is served as the baseline.

Dataset	PSMEKL	EKM	MEKL	Deepwalk	Node2vec	TAWE	Degree	RARW	Eigen
CORA	**71.2**$_{\pm 0.9}$	67.0$_{\pm 1.7}$	67.2$_{\pm 1.9}$	66.2$_{\pm 1.7}$	65.2$_{\pm 1.4}$	66.7$_{\pm 0.8}$	64.0$_{\pm 1.1}$	66.3$_{\pm 1.8}$	66.8$_{\pm 1.9}$
CiteSeer	**48.8**$_{\pm 1.1}$	45.4$_{\pm 1.1}$	44.7$_{\pm 2.0}$	42.0$_{\pm 2.5}$	42.6$_{\pm 1.2}$	43.7$_{\pm 1.9}$	40.4$_{\pm 1.6}$	44.1$_{\pm 1.5}$	47.3$_{\pm 0.6}$
PubMed	**71.9**$_{\pm 1.8}$	68.4$_{\pm 2.1}$	68.9$_{\pm 1.5}$	59.4$_{\pm 1.7}$	57.8$_{\pm 1.8}$	64.0$_{\pm 1.7}$	63.9$_{\pm 1.1}$	63.6$_{\pm 1.1}$	65.5$_{\pm 1.5}$
Brazil-air	**49.4**$_{\pm 1.6}$	47.4$_{\pm 2.5}$	47.7$_{\pm 2.7}$	48.1$_{\pm 4.9}$	48.1$_{\pm 1.5}$	48.7$_{\pm 2.0}$	47.6$_{\pm 2.5}$	48.9$_{\pm 1.3}$	47.7$_{\pm 2.1}$
Europe-air	**26.3**$_{\pm 6.2}$	23.5$_{\pm 2.1}$	21.6$_{\pm 2.6}$	22.1$_{\pm 2.7}$	21.8$_{\pm 2.5}$	**26.3**$_{\pm 6.0}$	23.8$_{\pm 7.9}$	25.0$_{\pm 4.0}$	21.4$_{\pm 2.5}$
USA-air	50.7$_{\pm 1.6}$	50.0$_{\pm 1.9}$	50.3$_{\pm 1.4}$	50.1$_{\pm 1.3}$	50.2$_{\pm 1.5}$	50.6$_{\pm 1.2}$	50.3$_{\pm 1.5}$	50.5$_{\pm 1.6}$	**51.2**$_{\pm 1.3}$
Wikipedia	39.6$_{\pm 1.3}$	40.9$_{\pm 1.2}$	40.5$_{\pm 0.9}$	**41.2**$_{\pm 0.7}$	40.6$_{\pm 1.4}$	40.6$_{\pm 0.9}$	39.0$_{\pm 4.3}$	**41.7**$_{\pm 0.7}$	41.1$_{\pm 1.1}$
BlogCatalog	12.9$_{\pm 1.5}$	13.1$_{\pm 1.2}$	13.0$_{\pm 1.1}$	**13.5**$_{\pm 0.7}$	13.0$_{\pm 1.7}$	13.3$_{\pm 0.5}$	13.1$_{\pm 0.9}$	**13.7**$_{\pm 0.6}$	13.2$_{\pm 0.8}$
Average	**46.4**	44.5	44.2	42.8	42.4	44.2	42.8	44.4	44.3

reported in GraphMAE (X. Liu et al., 2022). Additionally, we compared PSMEKL with other attribute-masking or edge-masking methods, including Nodeprop and DropEdge (W. Huang et al., 2020). We directly copy the results reported in the corresponding paper.

Basic Setting: In the experiment, we adopt GCN and GraphSAGE (Hamilton et al., 2017) as basic model to test the effectiveness of all embedding algorithms. Table 3 shows the detailed setting of the used hyper-parameters. The selection of α and β) should satisfy $\alpha + \beta = 1$. In experiments on non-attributed graphs, the number of GNN layers is chosen from $\{10, 15\}$, while for attributed graphs, the number of GNN layers is set to 2. Friedman test (Hollander et al., 2013) is adapted to validate the difference between the proposed PSMEKL and compared algorithms.

For the GCN and GraphSAGE, we use Adam (Kingma & Ba, 2015) as the optimizer. Each hyper-parameter configuration is tested 10 times, and the results are averaged to obtain the final outcome. We experiment with all parameter combinations and select the best one based on validation set performance to predict the test nodes. During training, we stop early if there is no improvement within 30 epochs. Each experiment on non-attributed graphs on every dataset is repeated 10 times. For attributed graphs, the repeated times on each dataset consist of the settings in GraphMAE (X. Liu et al., 2022).

4.1. Experiment performance on non-attributed graphs

This experiment demonstrates that PSMEKL outperform existing node embedding algorithm on non-attributed graphs. The detailed classification results on non-attributed graphs are shown in Tables 5 and 6, and the statical results are shown in Fig. 3.

From Tables 5 and 6, PSMEKL significantly improves the performance and outperforms the comparison algorithms on non-attributed graphs. Compared with EKM and MEKL, PSMEKL demonstrates superior performance due to its incorporation of both positional and structural information. When evaluated against DeepWalk and Node2Vec, kernel based methods achieve better results by considering more comprehensive node relationships through kernel matrix computation. Although TAWE and RARW significantly outperform DeepWalk and Node2Vec by incorporating structural information into random walks, their overall performance remains inferior to PSMEKL.

From the two tables, the average performance of PSMEKL indicates that it consistently outperforms other comparison algorithms. The Friedman tests, shown in Fig. 3, clearly illustrate that PSMEKL achieves the highest ranking, followed closely by kernel-related methods. Thus, we can conclude that the kernel-based methods, particularly in PSMEKL, excel at providing node embedding features with better discrimination.

4.2. Ablation study on non-attributed graphs

This experiment demonstrates that the effectiveness of the combination of both positional information and structural information achieves the best performance. The detailed results are shown in Table 7.

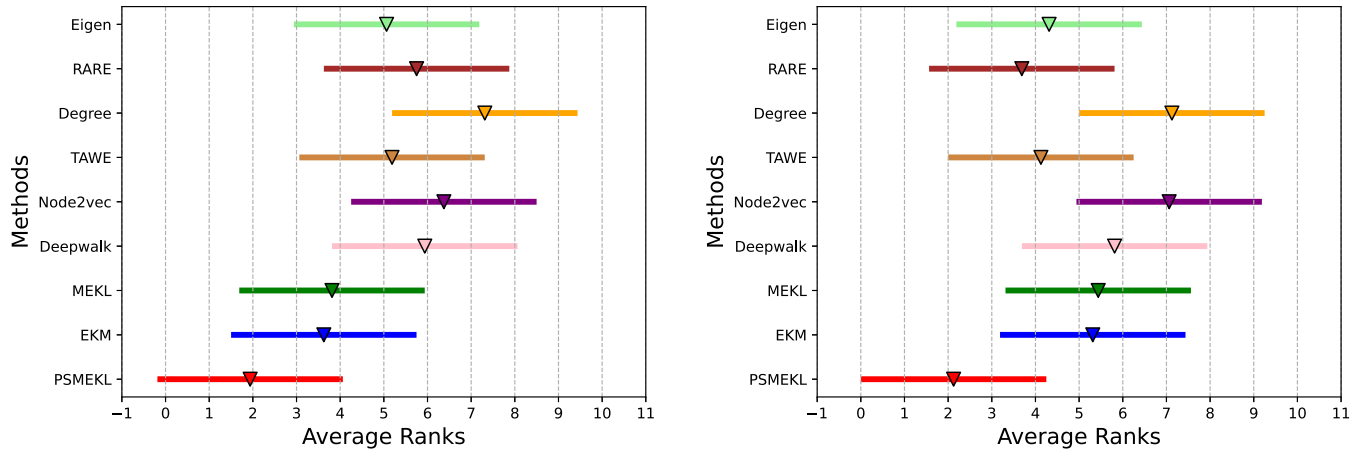
PSMEKL contains two important parameters α and β, controlling the importance of positional information and structural information, respectively. When $\alpha = 0$ and $\beta = 1$, the objective function focuses on community structure learning. When $\alpha = 1$ and $\beta = 0$, the objective function emphasizes consistent learning related to node position across different kernel spaces. Setting both α and β to 0.5 ensures the objective function equally considers consistent learning and community structure learning.

Table 7 show that the objective function focused on community structure learning outperforms that focused on consistent learning. However, the best performance is achieved when the objective function combines consistent learning and community structure learning. Thus, we can conclude that incorporating both types of information further improves performance.

4.3. Embedding efficiency on non-attributed graphs

This experiment shows that PSMEKL is more efficient than other node embedding algorithms for providing node embeddings on non-attributed graphs. Table 8 details the time required by each node embedding algorithm. In the table, we implemented DeepWalk and Node2vec using the $node2vec$ and $word2vec$ packages.

From the table, PSMEKL has the fastest training speed in experiments with non-attributed graphs. It's worth noting that PSMEKL optimizes the objective function with positional and structural information, and provides node embeddings with specified dimensions, while EKM and



(a) GCN (b) GraphSAGE

Fig. 3. The left and right figures reflect the visualized Friedman test for PSMEKL with GCN and GraphSAGE, respectively. (Closer to the left indicates better performance.)

Table 7
Results of varying α and β for PSMEKL using GCN and GraphSAGE. (The best result on each data set is written in bold).

GCN Datasets	$\alpha=1$ $\beta=0$	$\alpha=0$ $\beta=1$	$\alpha=0.5$ $\beta=0.5$	GraphSAGE Dataset	$\alpha=1$ $\beta=0$	$\alpha=0$ $\beta=1$	$\alpha=0.5$ $\beta=0.5$
CORA	$68.6_{\pm 1.6}$	$71.5_{\pm 0.9}$	$\mathbf{71.9}_{\pm 0.9}$	CORA	$66.5_{\pm 1.3}$	$\mathbf{71.2}_{\pm 0.9}$	$71.0_{\pm 1.3}$
CiteSeer	$46.2_{\pm 1.1}$	$\mathbf{50.7}_{\pm 1.7}$	$49.7_{\pm 1.0}$	CiteSeer	$44.8_{\pm 1.0}$	$48.6_{\pm 1.6}$	$\mathbf{48.8}_{\pm 1.1}$
PubMed	$66.1_{\pm 0.7}$	$69.3_{\pm 2.0}$	$\mathbf{69.6}_{\pm 1.7}$	PubMed	$63.5_{\pm 1.6}$	$\mathbf{71.9}_{\pm 1.8}$	$68.3_{\pm 0.7}$
Brazil-air	$41.1_{\pm 2.4}$	$39.1_{\pm 4.6}$	$\mathbf{43.0}_{\pm 0.3}$	Brazil-air	$48.3_{\pm 3.2}$	$49.1_{\pm 3.0}$	$\mathbf{49.4}_{\pm 1.9}$
Europe-air	$26.6_{\pm 0.6}$	$\mathbf{30.0}_{\pm 8.4}$	$29.6_{\pm 6.0}$	Europe-air	$21.6_{\pm 3.0}$	$21.2_{\pm 2.6}$	$\mathbf{26.3}_{\pm 6.2}$
USA-air	$54.0_{\pm 3.0}$	$\mathbf{54.1}_{\pm 1.0}$	$54.0_{\pm 1.3}$	USA-air	$\mathbf{51.0}_{\pm 1.0}$	$50.7_{\pm 1.6}$	$50.5_{\pm 1.6}$
Wikipedia	$\mathbf{44.6}_{\pm 0.9}$	$44.5_{\pm 1.2}$	$44.5_{\pm 0.8}$	Wikipedia	$39.3_{\pm 1.6}$	$39.6_{\pm 1.3}$	$\mathbf{39.6}_{\pm 0.9}$
BlogCatalog	$\mathbf{23.3}_{\pm 0.6}$	$22.0_{\pm 0.7}$	$21.9_{\pm 0.7}$	BlogCatalog	$12.9_{\pm 1.5}$	$13.3_{\pm 1.0}$	$\mathbf{13.7}_{\pm 0.5}$
Average	46.3	47.7	**48.0**	Average	43.5	45.7	**46.0**

Table 8
Required training times of node embedding algorithms are listed. (The best result is written in bold).

Algorithm	Wikipedia	BlogCatalog	PubMed
PSMEKL	**4.4 (s)**	**54.9 (s)**	**252.7 (s)**
EKM	7.4 (s)	393.5 (s)	2179.9 (s)
Eigen	6.5 (s)	305.5 (s)	1706.3 (s)
Deepwalk	322.6 (s)	4145.6 (s)	1754.1 (s)
Node2vec	125.2 (s)	10,520.3 (s)	2063.9 (s)

Eigen produces embeddings with dimensions that approximate the number of nodes. Movoer, algorithms like EKM and Eigen perform eigen-decomposition on the entire graph, giving them no efficiency advantage. Additionally, DeepWalk and Node2vec take a long time to calculate node embeddings, especially as the number of edges increases.

In summary, PSMEKL has advantages in embedding efficiency for non-attributed graphs. In addition to the training time results, Fig. 4 shows the loss function curve. The figure reveals that the loss values drop rapidly within the first 10 steps, with similar trends observed for both attributed and non-attributed graphs.

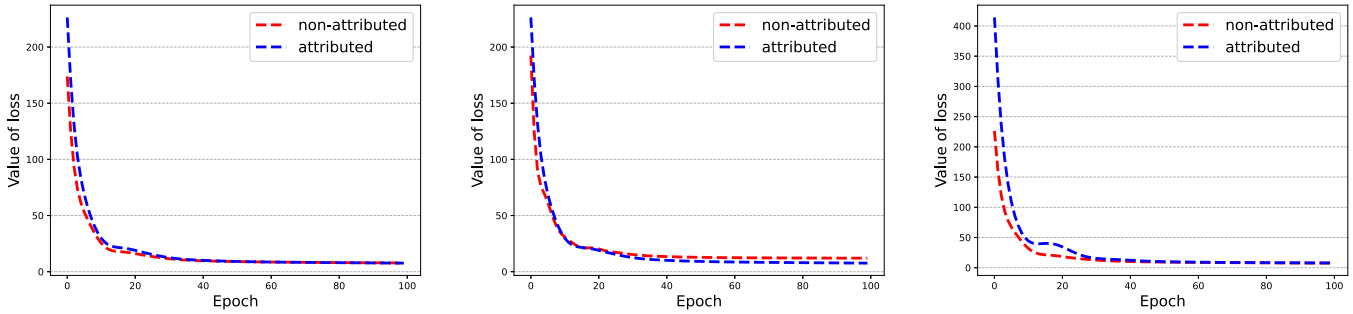
4.4. Experiment performance on attributed graphs

This experiment shows that PSMEKL significantly improves the performance and outperforms the comparison algorithms on attributed graphs. The detailed classification results are presented in Table 9, with the results of comparison algorithms directly copied from their respective papers.

Table 9
Acc (%) results of comparison algorithms on attributed graphs are reported here. (ORG stands for the original node features. The best result on each data set is written in bold.)

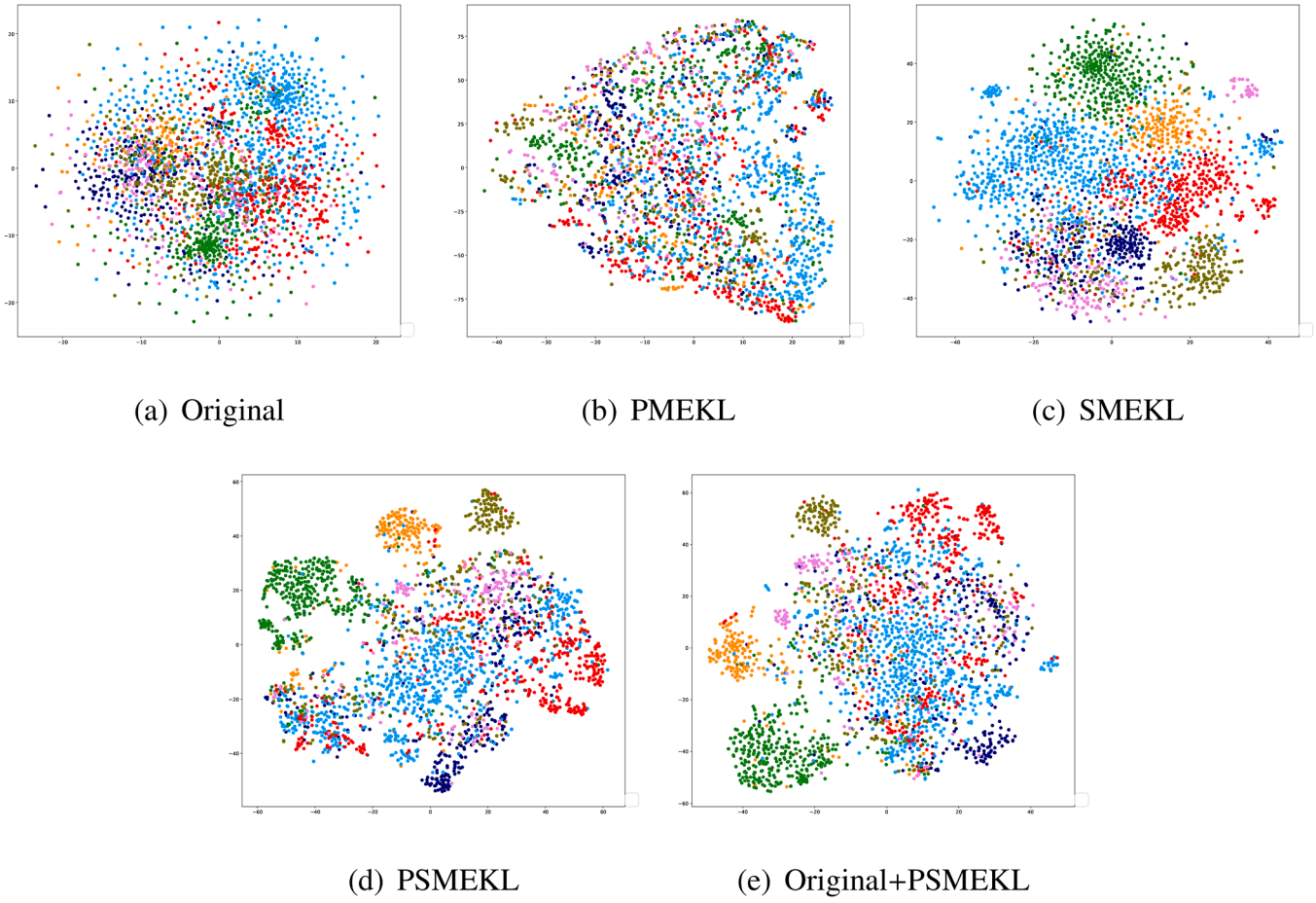
Algorithm	CORA	CiteSeer	PubMed
GCN (baseline)	$81.7_{\pm 0.5}$	$70.2_{\pm 0.5}$	$79.4_{\pm 0.3}$
DropEdge	82.8	72.3	79.6
NodeProp	81.9	71.6	79.4
GATE	$83.2_{\pm 0.6}$	$71.8_{\pm 0.8}$	$80.9_{\pm 0.3}$
BGRL	$82.7_{\pm 0.6}$	$71.1_{\pm 0.7}$	$79.4_{\pm 0.6}$
CCA-SSG	$84.0_{\pm 0.4}$	$\mathbf{73.1}_{\pm 0.3}$	$81.0_{\pm 0.4}$
PACER	$82.1_{\pm 0.9}$	$70.6_{\pm 0.9}$	$79.1_{\pm 1.1}$
GraphMAE	$82.9_{\pm 0.6}$	$72.5_{\pm 0.5}$	$81.0_{\pm 0.5}$
PSMEKL	$82.9_{\pm 0.7}$	$71.5_{\pm 1.2}$	$\mathbf{81.7}_{\pm 1.2}$
ORG + PSMEKL	$\mathbf{84.5}_{\pm 0.6}$	$72.7_{\pm 0.7}$	$\mathbf{81.7}_{\pm 0.7}$

Moreover, the detailed classification results on attributed graphs are presented in Table 9, with the results of comparison algorithms directly copied from their respective papers. From the table, we observe that PSMEKL achieves an approximate improvement of 2.5% on the CORA, CiteSeer, and Pubmed datasets. Since PSMEKL provides embedding features through pre-processing, it is easy to combine PSMEKL embeddings with the original features. As the results show, the combination of



(a) Cora (b) Citeseer (c) Pubmed

Fig. 4. The convergence curve of the proposed PSMEKL is displayed for attributed graphs (blue dashed line) and non-attributed graphs (red dashed line).



(a) Original (b) PMEKL (c) SMEKL

(d) PSMEKL (e) Original+PSMEKL

Fig. 5. The visualized distribution of nodes in different classes, represented by different colors, on the CORA dataset.

Table 10
Micro-F11% scores of all used algorithms on large-scale graph datasets are listed in the table. (The best result is written in bold and the second-best result are underlined. NA stands for being not available.).

Method	Position Info	Structure Info	PPI	Flickr	Reddit	Yelp	Amazon
Baseline	✗	✗	95.93	50.67	95.04	60.12	72.37
PSMEKL	✓	✗	96.16	50.76	95.12	**60.42**	**76.95**
PSMEKL	✓	✓	**96.28**	**51.12**	**95.22**	60.37	NA

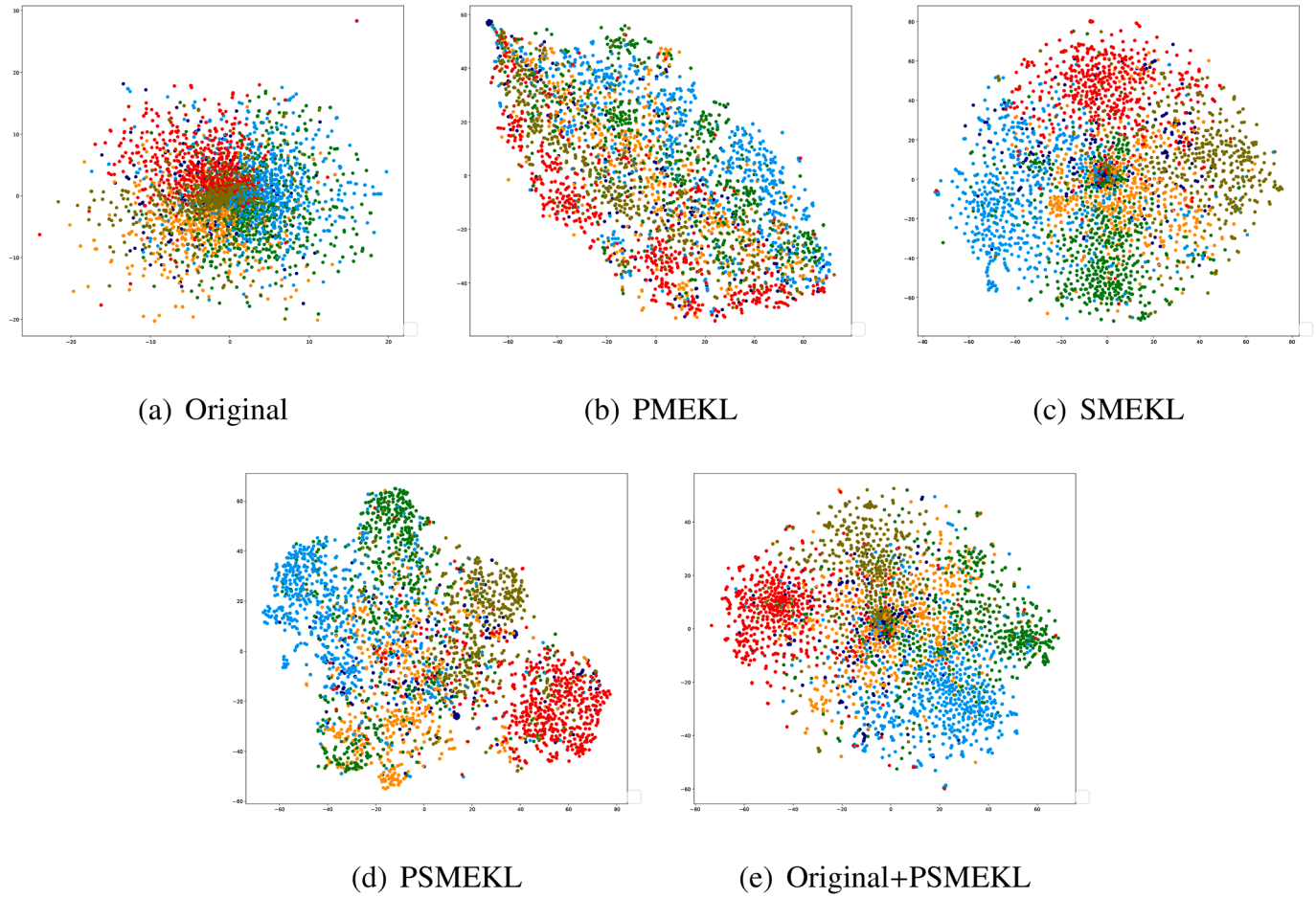


Fig. 6. The visualized distribution of nodes in different classes, represented by different colors, on the CiteSeer dataset.

original feature (ORG) and PSMEKL further improves performance and achieves the best results on CORA and PubMed datasets.

Table 10 presents the experimental results of PSMEKL after incorporating positional and structural information, respectively. While large-scale graph data poses challenges for kernel mapping, PSMEKL remains operational through selective node sampling for kernel matrix construction. In the experiment, we select SIGN (E. Rossi et al., 2020) as the baseline method to reduce the memory consuming.

From Table 10, experimental results demonstrate that non-augmented node data yields the poorest classification performance. Moreover, classification accuracy shows additional gains after incorporating explicit kernel mapping for node position representation. Furthermore, the introduction of positional and structural information consistently improves results on PPI, Flickr, and Reddit datasets. When running on the Yelp dataset, the community discovery algorithm obtained over 25,000 communities, and excessive community classification may hinder the accuracy of the classification results. Therefore, on the Yelp dataset, the experimental results showed a slight decrease.

The primary bottleneck lies in the community detection algorithm, which extracts excessive communities from large graph structures. Moreover, it fails to complete community discovery tasks for graphs exceeding one million nodes. From Table 10, although Louvain serves as an efficient community detection algorithm, it tends to generate excessive communities and may become inoperable on large-scale graph data. For instance, on the Yelp dataset, Louvain produces approximately 25,000 communities, which adversely affects community classification. Furthermore, on the Amazon dataset, the community detection algorithm fails to execute due to the overwhelming number of nodes.

4.5. Visualized results of PSMEKL

In the experiment, we use t-SNE (L. J. P. van der Maaten, 2008) to visualize the results of PSMEKL on CORA, CiteSeer, and PubMed in Figs. 5–7. In these figures, Original stands for the original features without any changes. PMEKL stands for the node features with positional information generated by multi-kernel mapping. SMEKL stands for node features with structural information generated by community detection. PSMEKL stands for node features with both positional and structural information. Original + PSMEKL contacts the node features of Original and PSMEKL.

In these figures, Sub-figure (a) shows that the original node features exhibit significant overlap and poor discrimination in the visualization, making node separation challenging. After introducing positional information via kernel mapping, sub-figure (b) shows the inter-node discrimination improves markedly, facilitating downstream classification tasks. With structural information incorporated through community detection, sub-figure (c) shows nodes from different categories become globally separable. Sub-figure (d) shows nodes endowed with both positional and structural information achieve dual discrimination between individual nodes and that in different classes. Combining original features with features generated by PSMEKL, Sub-figure (e) induces controlled boundary ambiguity between different classes, enhancing model generalization.

The evolution of node distributions in the figures demonstrates that positional information introduced through kernel mapping clarifies local positional relationships between nodes. Structural information incorporated via community detection enables nodes to cluster according to structural patterns. Consequently, the simultaneous integration of positional and structural information yields clearly separable boundaries

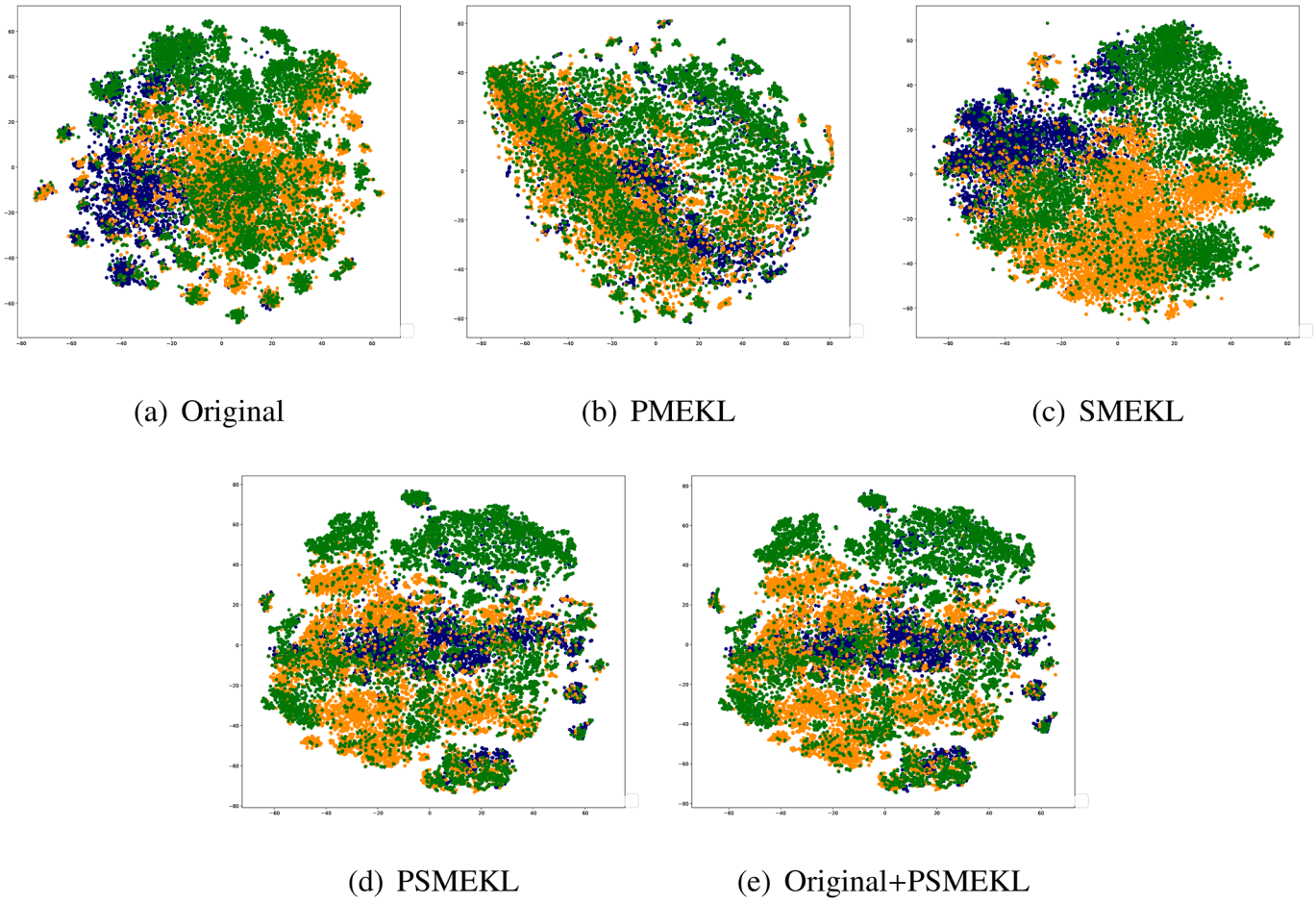


Fig. 7. The visualized distribution of nodes in different classes, represented by different colors, on the PubMed dataset.

between different node classes. However, this approach may carry certain overfitting risks. Experimental results indicate that model performance improves when combining these features with original attributes to achieve moderately blurred class boundaries.

According to the Rademacher complexity, a smaller hypothesis space corresponds to a lower Rademacher complexity, resulting in a tighter generalization error bound and thus better model generalization performance. Compared to scenarios with clear boundaries, ambiguous boundaries introduce data overlap, which constrains the feasible solution space. This reduces the corresponding hypothesis space, yielding tighter generalization bounds and superior generalization capability.

5. Conclusion

This work underscores the importance of preprocessing node features before aggregation in Graph Neural Networks (GNNs) to enhance performance on downstream tasks. Unlike conventional GNNs that rely primarily on topological links, the proposed Positional and Structural Multiple Empirical Kernel Learning (PSMEKL) enriches node embeddings by integrating positional and structural information. By combining multi-kernel mapping with community detection, PSMEKL generates robust embeddings for both attributed and non-attributed graphs. This method strengthens the differentiation of nodes with distinct structures while preserving similar embeddings for nodes with shared connectivity patterns, thus enhancing feature representations and improving GNN model performance. Experimental results consistently demonstrate its effectiveness in improving the quality of node representations, thereby boosting GNN performance across diverse datasets.

Although the reliance on eigen-decomposition introduces a computational complexity of $O(N^3/P^2)$ for PSMEKL, making it resource-intensive for large-scale graphs despite partitioning strategies, we prioritize high-influence nodes to form the kernel matrix, which drastically reduces computational complexity, to improve efficiency and scalability. However, in terms of community discovery, although Louvain serves as an efficient community detection algorithm, it tends to generate excessive communities, which adversely affects community detection and classification, and may become inoperable on large-scale graph data. Future work will focus on developing community detection algorithms specifically tailored to large-scale graph data.

CRediT authorship contribution statement

Zonghai Zhu: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Funding acquisition, Formal analysis; **Xinshuai Wei:** Resources, Methodology, Investigation, Data curation; **Huanlai Xing:** Validation, Supervision, Funding acquisition; **Li Feng:** Validation, Supervision, Funding acquisition; **De Chen:** Validation, Supervision, Funding acquisition; **Yuge Xu:** Visualization, Validation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by Sichuan Science and Technology Program (2024NSFSC1472), Natural Science Foundation of China under Grant No. 62473159, and Science and Technology Innovation Team Project of the Special Fund of Basic Scientific Research Business Fund of Central Universities of the Southwest Jiaotong University, Grant/Award Number: 2682022JX008.

References

Blondel, V. D., Guillaume, J. L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics Theory and Experiment*, *10*, P10008.

Both, C., Dehmamy, N., Yu, R., & Barabási, A. L. (2023). Accelerating network layouts using graph neural networks. *Nature Communications*, *14* (1), 1560.

Brin, S., & Page, L. (2012). Reprint of: The anatomy of a large-scale hypertextual web search engine. *Computer Networks*, *56*(18), 3825–3833.

C. Tallec, S. T., Azar, M. G., Azabou, M., Dyer, E. L., Munos, R., Velickovic, P., & Valko, M. (2022). Large-scale representation learning on graphs via bootstrapping. In *International conference on learning representations*.

C. Wang, C. W., Xu, J., Liu, Z., Zheng, K., Wang, X., Song, Y., & Gai, K. (2023). Graph contrastive learning with generative adversarial network. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 2721–2730).

C. You, A. F., Wang, S., & Tassiulas, L. (2022). KerGNNs: Interpretable graph neural networks with graph kernels. In *Association for the advance of artificial intelligence* (pp. 6614–6622).

Chen, X., Chen, S., Yao, J., Zheng, H., Zhang, Y., & Tsang, I. W. (2022). Learning on attribute-missing graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *44* (2), 740–757.

D. Mandaglio, L. Z., & Tagarelli, A. (2024). Link prediction on multilayer networks through learning of within-layer and across-layer node-pair structural features and node embedding similarity. In *ACM on web conference* (pp. 924–935).

Dey, A., Kumar, B. R., Das, B., & Ghoshal, A. K. (2023). Outlier detection in social networks leveraging community structure. *Information Sciences*, *634*, 578–586.

E. Rossi, F. F., Eynard, D., Chamberlain, B., Bronstein, M., & Monti, F. (2020). Sign: Scalable inception graph neural networks. In *ICML 2020 workshop on graph representation learning and beyond*.

G. Kou, H. Z., Peng, Y., & Zhang, B. (2024). Role-aware random walk for network embedding. *Information Sciences*, *652*, 119765. https://doi.org/10.1016/J.INS.2023.119765

G. Xu, W. Z., Cui, Z., Luo, S., Long, C., & Zhang, T. (2023). Deep graph structural infomax. In *AAAI Conference on artificial intelligence* (pp. 4920–4928).

H. Davulcu, A. S. (2020). Graph attention auto-encoders. In *IEEE International conference on tools with artificial intelligence* (pp. 989–996).

H. Xing, Z. Z., & Xu, Y. (2023). Balanced neighbor exploration for semi-supervised node classification on imbalanced graph data. *Information Sciences*, *631*, 31–44.

Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in neural information processing systems* (pp. 1024–1034).

Hirchoua, B., & El Motaki, S. (2023). -Random walk: Collaborative sampling and weighting mechanisms based on a single parameter for node embeddings. *Pattern Recognition*, *142*, 109730. https://doi.org/10.1016/j.patcog.2023.109730

Hollander, M., Wolfe, D., & Chicken, E. (2013). Nonparametric statistical methods. John Wiley & Sons.

J. Leskovec, A. G. (2016). Node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 855–864).

J. Li, Q. Z., Sun, Y., Wang, S., Gao, J., & Yin, B. (2024). Beyond low-pass filtering on large-scale graphs via adaptive filtering graph neural networks. *Neural Networks*, *169*, 1–10.

J. You, Y. C., He, J., Lin, Y., Peng, Y., Wu, C., & Zhu, Y. (2023). SP-GNN: Learning structure and position information from graphs. *Neural Networks*, *161*, 505–514.

J. Zhang, L. G., Tang, L., Chen, T., Zhu, L., & Yin, H. (2024). Time interval-enhanced graph neural network for shared-account cross-domain sequential recommendation. *IEEE Transactions on Neural Networks and Learning Systems*, *35* (3), 4002–4016.

K. Liang, G. Z., Pan, C., Zhang, F., Wu, X., Hu, X., & Yu, Y. (2023). Graph convolution based cross-network multiscale feature fusion for deep vessel segmentation. In *IEEE transactions on medical imaging* (pp. 183–195). (*vol. 42*).

Kingma, D., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International conference on learning representations*.

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations*.

Li, Y., & Shawe-Taylor, J. (2006). Using KCCA for Japanese-English cross-language information retrieval and document classification. *Journal of Intelligent Information Systems*, *27* (2), 117–133.

Liu, X., Zhang, F., Hou, Z., Mian, L., Wang, Z., Zhang, J., & Tang, J. (2023). Self-supervised learning: Generative or contrastive. *IEEE Transactions on Knowledge and Data Engineering*, *35* (1), 857–876.

L. J. P. van der Maaten, G. E. H. (2008). Visualizing high-dimensional data using t-SNE. *Journal of Machine Learning Research*, *9*, 2579–2605.

Pizzuti, C. (2018). Evolutionary computation for community detection in networks: A review. *IEEE Transactions on Evolutionary Computation*, *22*(3), 464–483.

Q. Chen, J. Q., Dong, Y., Zhang, J., Yang, H., Ding, M., Wang, K., & Tang, J. (2020). GCC: Graph contrastive coding for graph neural network pre-training. In *Proceedings of the 26th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 1150–1160).

Q. Wu, H. Z., Yan, J., Wipf, D., & Yu, P. S. (2021). From canonical correlation analysis to self-supervised graph neural networks. In *Advances in neural information processing systems* (pp. 76–89).

R. Al-Rfou, B. P., & Skiena, S. (2014). Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 701–710).

Rawat, W., & Wang, Z. (2017). Deep convolutional neural networks for image classification: A comprehensive review. *Neural Computation*, *29*(9), 2352–2449.

Saha, S., Gao, J., & Gerlach, R. (2022). A survey of the application of graph-based approaches in stock market analysis and prediction. *International Journal of Data Science and Analytics*, *14*(1), 1–15.

Shawe-Taylor, J., & Cristianini, N. (2004). Kernel method for pattern analysis. *Journal of the American Statistical Association*, *101* (476), 1730.

Shen, X., Ong, Y. S., Mao, Z., Pan, S., Liu, W., & Zheng, Y. (2023). Compact network embedding for fast node classification. *Pattern Recognition*, *136*, 109236. https://doi.org/10.1016/j.patcog.2022.109236

Su, X., Xue, S., Liu, F., Wu, J., Yang, J., Zhou, C., Hu, W., Paris, C., Nepal, S., Jin, D., Sheng, Q., & Yu, P. S. (2022). A comprehensive survey on community detection with deep learning. *IEEE Transactions on Neural Networks and Learning Systems*, *35* (4), 4682–4702.

V. C. Ta, T. L. H. (2024). Balancing structure and position information in graph transformer network with a learnable node embedding. *Expert Systems with Applications*, *238*, 122096. https://doi.org/10.1016/J.ESWA.2023.122096

W. Huang, Y. R., Xu, T., & Huang, J. (2020). DropEdge: Towards deep graph convolutional networks on node classification. In *International conference on learning representations*.

Wu, S., Sun, F., Zhang, W., Xie, X., & Cui, B. (2022). Graph neural networks in recommender systems: A survey. *ACM Computing Surveys*, *55* (5), 1–37.

X. Chen, Y. F., Xu, S., Li, J., Yao, X., Huang, Z., & Wang, Y. (2025). GSSCL: A framework for graph self-supervised curriculum learning based on clustering label smoothing. *Neural Networks*, *181*, 106787.

X. Liu, H. T., & Murata, T. (2021). Graph convolutional networks for graphs containing missing features. *Future Generations Computer Systems : FGCS*, *117*, 155–168.

X. Liu, Z. H., Cen, Y., Dong, Y., Yang, H., Wang, C., & Tang, J. (2022). GraphMAE: Self-supervised masked graph autoencoders. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 594–604).

Xiong, H., Swamy, M. N. S., & Ahmad, M. O. (2005). Optimizing the kernel in the empirical feature space. *IEEE Transactions on Neural Networks*, *16* (2), 74–460.

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? In *International conference on learning representations*.

Y. Fang, D. B., Liu, Y., & Shi, C. (2023). Graph contrastive learning with stable and scalable spectral encoding. In *Advances in neural information processing systems*.

Y. Hu, Y. Y., Zhou, Q., Liu, L., Zeng, Z., Chen, Y., Pan, M., Chen, H., Das, M., & Tong, H. (2024a). PaCEr: Network embedding from positional to structural. In *ACM on web conference, 2024* (pp. 2485–2496).

Y. Hu, Y. Y., Zhou, Q., Wu, S., Wang, D., & Tong, H. (2024b). Topological anonymous walk embedding: A new structural node embedding approach. In *International conference on information and knowledge management* (pp. 2796–2806).

Y. Xu, Y. Z., Yu, F., Liu, Q., Wu, S., & Wang, L. (2021). Graph contrastive learning with adaptive augmentation. In *Proceedings of the ACM web conference* (pp. 2069–2080).

Ye, J., Zhao, J., Ye, K., & Xu, C. (2020). How to build a graph-based deep learning architecture in traffic domain: A survey. *IEEE Transactions on Intelligent Transportation Systems*, *23* (5), 3904–3924.

Z. Lu, H. C., Li, P., & Yang, C. J. (2022). On positional and structural node features for graph neural networks on non-attributed graphs. In *ACM international conference on information and knowledge management* (pp. 3898–3902).

Z. Wang, Z. Z., Li, D., & Du, W. (2019). Multiple empirical kernel learning with majority projection for imbalanced problems. *Applied Soft Computing*, *76*, 221–236.